How PINNs scale on multi-soliton problems, and what that tells us about what comes next

Soham Pujari, Nirek Agarwal
BITS Pilani, K K Birla Goa Campus

June 21, 2026

Abstract

PINNs work well on the single-soliton KdV problem. We asked a simple follow-up: how far does that hold up as we add more solitons? We trained PINNs on N-soliton configurations from N=3 up to N=12, keeping the architecture and training recipe fixed. From N=3 to N=9, scaling was smooth and almost boring. The error grew roughly linearly with N. At N=12 the story broke. Errors jumped by an order of magnitude and the L-BFGS optimizer started making things worse rather than better. We spent the rest of the study figuring out why. The N=12 failure turned out not to be a capacity problem. Four different interventions independently rescued the run, and the cheapest one by far was adding a small amount of supervised data. When we ran the same N=12 configuration with eight different random seeds, we got six different failure patterns, and two seeds essentially rescued themselves. The wall was not really a wall. It was a probabilistic barrier with multiple partial solutions competing with the correct one. The optimizer landed in one of them unless something biased it toward the right basin.

But the KdV study was never the end goal. The b-family of shallow-water equations, particularly Camassa-Holm and Degasperis-Procesi, admit non-smooth wave solutions called peakons and, in the DP case, shockpeakons. These are harder problems in every sense, and simulating the shockpeakon remains an open problem. Everything we did here, the diagnostic ladder, the scaling sweep, the per-feature breakdown, the minimum-intervention map, the multi-seed test, was designed with that target in mind. We now know what smooth PINN scaling looks like, what landscape-driven failure looks like, and what the cheapest fix is. That is the foundation the b-family work will build on.

Code: github.com/sohp2005/KdV-solution-using-PINNs check under multi_soliton_extended/

1 Where we started, and why : RECAP

Our earlier work in this project looked at the single-soliton KdV problem with PINNs. We characterized how the network learned a single moving bump, studied noise robustness, and benchmarked the Adam to L-BFGS schedule that has become standard in the PINN literature. That study left us with a clean baseline. PINNs can solve single-soliton KdV to under 0.1% error with a small network and a simple training recipe. The natural next question was what happens when we put more than one soliton in the box.

The KdV equation describes shallow water waves. Specifically, it describes waves that don't slow down, don't spread out, and don't change shape. The kind of wave John Scott Russell chased on horseback along a Scottish canal in 1834 because he couldn't believe what he was seeing. These

waves are called solitons. The equation that predicts them is

$$u_t + 6uu_x + u_{xxx} = 0. \tag{1}$$

A single soliton looks like a clean bump moving to the right at speed c, given exactly by

$$u(x,t) = \frac{c}{2}\,\mathrm{sech}^2\left[\frac{\sqrt{c}}{2}(x - ct - x_0)\right]. \tag{2}$$

The faster the soliton, the taller and sharper it is. Two solitons pass through each other and emerge unchanged, a property almost unique to the KdV equation, and one of the reasons solitons are considered one of the more beautiful objects in mathematical physics.

PINNs are a relatively recent idea. Instead of solving a PDE on a grid, we train a neural network to satisfy the equation at a sample of points, while also matching initial conditions, boundary conditions, and a handful of supervised data points. For a single KdV soliton, they work very well. The natural next question is what happens when we try to fit more than one soliton at a time.

That is where this paper begins.

1.1 What we actually did

We trained PINNs on N-soliton configurations, stepping N from 3 to 12. For each N, we kept everything else fixed. Same architecture, six layers of 50 neurons each with tanh activation. Same optimizer schedule, 15,000 steps of Adam followed by 2,000 steps of L-BFGS. Same loss weights, same recipe for how many training points to use. We just asked whether it still worked.

The answer surprised us.

2 How we set things up

2.1 The multi-soliton reference

We use a linear superposition of single solitons as our ground truth. This is an approximation. The exact N-soliton solution from inverse scattering theory is more involved, but for well-separated solitons with different speeds the superposition is very accurate. We set up initial positions so that solitons don't overlap much at $t = 0$, and we use a domain wide enough that each soliton has room to move without hitting a boundary.

As N grows, two things grow together. The domain gets wider to fit more solitons, and the speed range widens to keep the solitons distinguishable. Higher-speed solitons are sharper and taller. At N=12, the fastest soliton has $c = 7.0$ and the slowest has $c = 1.5$.

Table 1: What changes as N grows.

N	Speed range	Domain width	Total training points (approx.)
3	1.5 to 2.5	58	19,000
6	1.5 to 4.0	95	31,000
9	1.5 to 5.5	132	44,000
12	1.5 to 7.0	170	59,000

2.2 The PINN itself

The network takes (x, t) as input and outputs u. The loss has four terms. The PDE residual measures how well the KdV equation is satisfied. The initial condition term measures how well we match the known t = 0 state. The boundary condition term checks values at the edges of the domain. The data term checks a small set of supervised values scattered through the domain. We weight all four equally and train with 15,000 Adam steps followed by 2,000 L-BFGS steps. We verify every training run against the exact solution to make sure our labels are correct, with a hard assertion in the code rather than just a visual check.

3 The smooth climb, then the cliff

3.1 N=3 to N=9: it just works

At N=3, the trained network matches the exact solution to 0.16% global L2 error. Barely visible. N=4 is 0.23%. N=6 is 0.49%. N=9 is 0.98%. Doubling the soliton count roughly doubles the error. Things degrade, but gracefully. By N=9, we have a nine-soliton solution with nine different wave speeds, learned to under 1% error, with nothing special done to make it work.

The L-BFGS step is worth noting here. Adam does most of the learning but tends to stall before reaching a precise minimum. L-BFGS then polishes the result, taking the error from say 3.9% down to 0.98% at N=9. It is a two-phase process that works cleanly at every N up to 9.

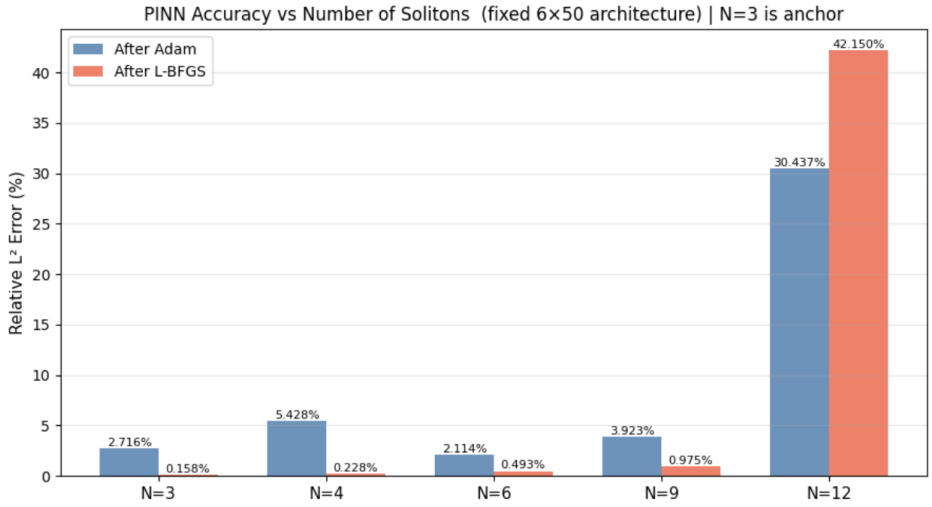

Figure 1: L2 error vs N on a log scale. Two curves, one after Adam and one after L-BFGS. The L-BFGS polish works cleanly up through N=9 and then breaks at N=12.

3.2 N=12: something breaks

At N=12, Adam stalls at 30.4%. Higher than at N=9, but not yet alarming. Then L-BFGS runs. And instead of polishing the result, it makes things worse. The error jumps to 42.2%.

That is the surprise. Not that N=12 is harder, which is expected, but that L-BFGS confidently makes things worse. L-BFGS is a second-order optimizer. It does not thrash randomly. When it moves, it moves in the direction it believes is downhill. If it walks us to 42% from 30%, it is because

it found a local minimum at 42% and was wrong about that being a good place to go.

Something qualitatively different is happening at N=12. The error is not mildly worse. It is ten times worse. And the failure is not random-looking. It is a confident wrong answer.

Table 2: The cliff at N=12.

N	After Adam	After L-BFGS	Did L-BFGS help?
3	2.72%	0.16%	yes, dramatically
6	2.11%	0.49%	yes, dramatically
9	3.92%	0.98%	yes, clearly
12	30.44%	42.15%	no, made it worse

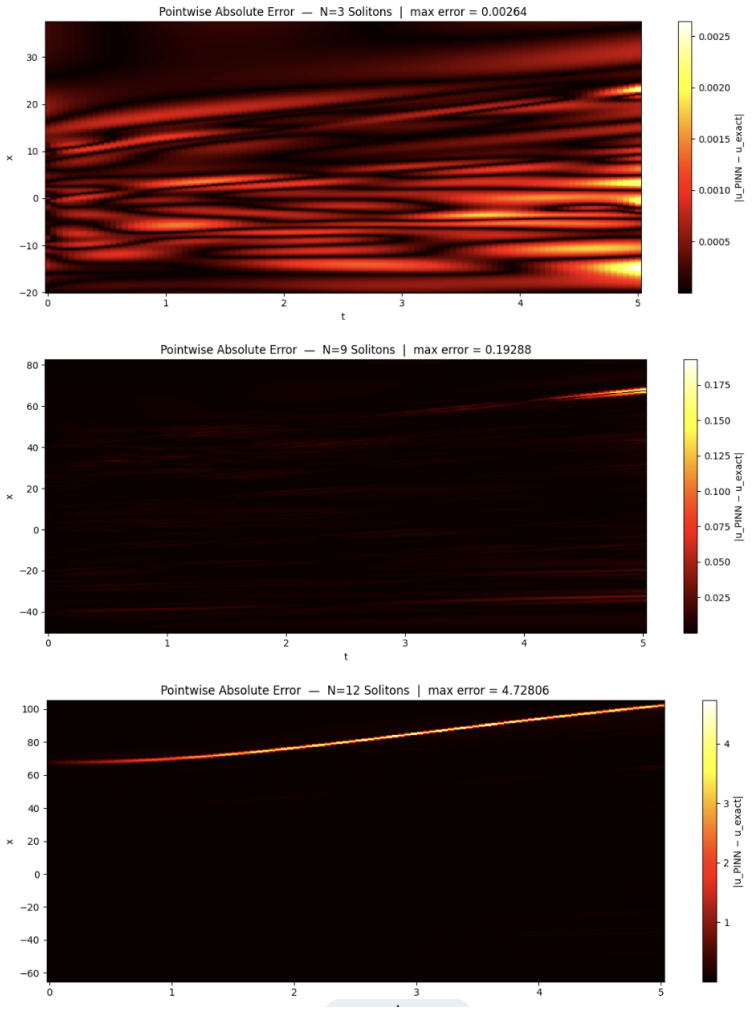

Figure 2: Solution heatmaps at N=3, N=9, and N=12. Predicted, exact, and error panels for each. The N=12 panels make the failure visible at a glance.

The rest of the paper is our attempt to understand why.

4 The first question: is the network too small?

When a neural network fails, the most natural first hypothesis is that it does not have enough capacity. Maybe six layers of 50 neurons is just not enough to represent twelve solitons at once. So we checked.

4.1 Making the network bigger

We ran N=12 with wider networks. First 75 neurons per layer, then 80, 85, 90, 95, and finally 100. If capacity is the problem, a wider network should steadily improve things.

Here is what we got.

Table 3: Width sweep at N=12. Capacity grows roughly fourfold from top to bottom.

Width	Parameters	Final error
50 (baseline)	13,000	42.15%
75	29,000	24.39%
80	33,000	39.11%
85	37,000	38.74%
90	41,000	32.64%
95	46,000	39.11%
100	51,000	0.86%

Two things stand out. First, width=100 suddenly drops to 0.86%. A fifty-fold improvement. That looks like capacity at first glance. But second, the path from 75 to 100 is not smooth. Width=80 is worse than width=75. Width=95 is worse than width=90. The error bounces up and down in a way that has nothing to do with capacity.

If this were a capacity problem, a network with 50,000 parameters would smoothly dominate one with 13,000 parameters. Instead, what we see is a switching pattern. The error flips between two regimes, roughly bad and catastrophic, with the jump between them sharp and unpredictable.

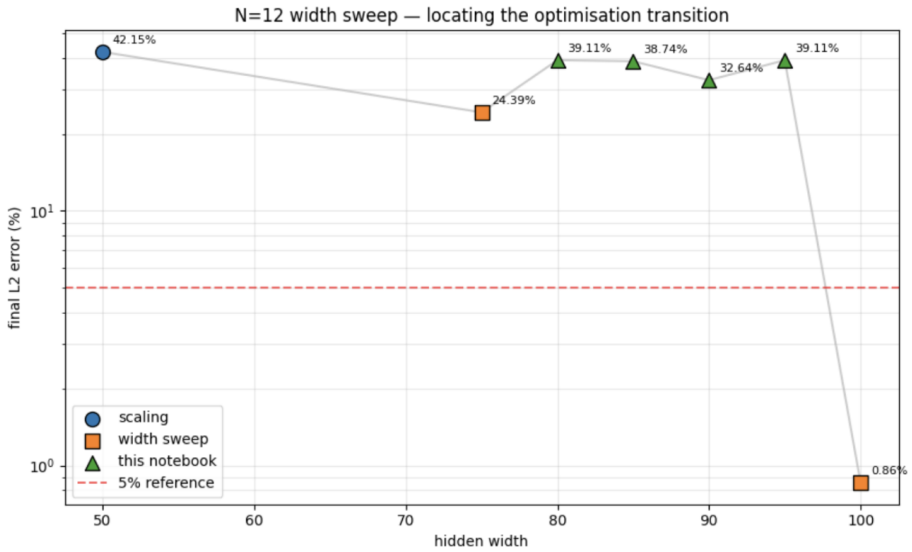

Figure 3: Final L2 error vs network width at N=12. The bouncing behaviour between widths 75 and 95 is the visual point. If this were a capacity problem, the trend would be monotonic.

A capacity ceiling produces smooth scaling. What we see looks more like a landscape problem. The optimizer is getting stuck in different places depending on the network size, not consistently finding a better solution.

We kept the width=100 result in the paper, but we want to be honest about it. When we reran the same configuration, we got 38.5% instead of 0.86%. The 0.86% number was not reliably reproducible. That is itself a clue.

4.2 Where exactly is the error?

The second observation came when we broke the N=12 error down by soliton. Instead of asking how wrong are we overall, we asked which solitons are wrong. For each soliton, we computed a local error in a window around its centre position, across all time steps.

Table 4: Per-soliton error at N=12 with width=50. Ten of twelve solitons are essentially solved. Two are catastrophically missed.

Soliton	Speed c	Local error
1	7.0	2.3%
2	6.5	86.7%
3	6.0	50.6%
4	5.5	7.0%
5 to 12	1.5 to 5.0	1 to 14%

Ten of twelve solitons are learned well. Two are catastrophically wrong, solitons 2 and 3, both with fast speeds, both positioned near the top of the domain. And when we repeat this breakdown for the width=100 run that gave 38.5% rather than 0.86%, we see the same signature. Solitons 2 and 3 are still the ones failing.

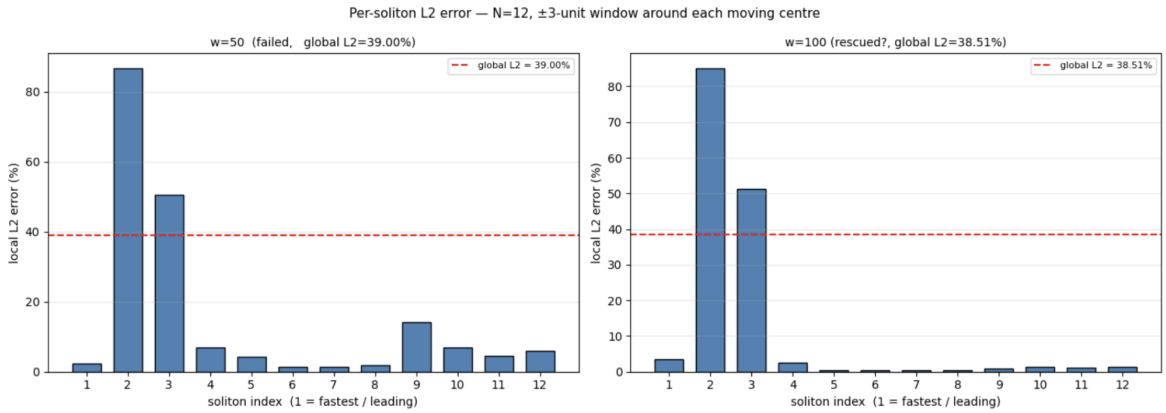

Figure 4: Per-soliton error breakdown for two different N=12 failure runs, one with width=50 and one with width=100. The bars for solitons 2 and 3 are dramatically larger than the others in both cases.

This rules out capacity. A network that cannot represent twelve solitons would fail everywhere. A network that perfectly represents ten of them and catastrophically misses two has a different problem. It has found a solution that is locally consistent for most of the domain but is locked onto the wrong configuration for those two solitons. The optimizer settled into what we will call a partial solution, internally consistent, confidently wrong, and wrong in a specific localized way.

5 What actually rescues it

Having ruled out capacity, we tried looking for what does work. If the network has enough capacity, there should be some way to make it use that capacity correctly. We tried four things.

5.1 More supervised data

We increased the number of supervised data points, the known correct values scattered through the domain. Starting from 1,133 in our default setup, we swept up from 0 to 2,000.

Table 5: Effect of supervised data on N=12. Same architecture, same optimizer, same everything else.

Data points	Final error
0	72.9%
100	81.2%
200	75.0%
500	1.95%
1000	3.86%
2000	1.22%

The jump at 500 points is dramatic. A 35-fold error reduction from just adding 500 supervised points. The capacity was always there. The network just needed to be told which solution to converge to.

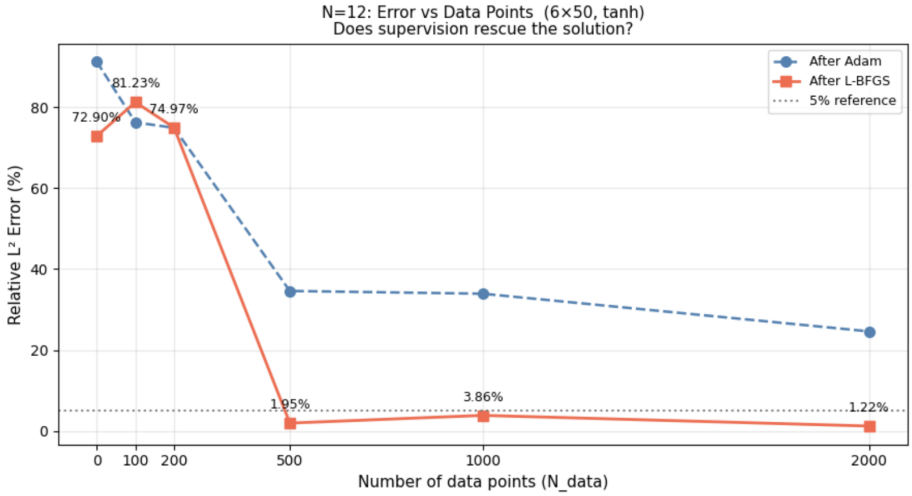

Figure 5: Final L2 error vs number of supervised data points at N=12, log scale on the y-axis. The cliff at 500 points is the story.

5.2 More collocation points

We also tried adding more PDE collocation points (FYI: the locations where the network is trained to satisfy the KdV equation is what I mean by that). This helps too, but much less efficiently.

Table 6: Effect of collocation density on N=12.

PDE points	Final error
20,000	30.6%
40,000	42.9%
80,000	24.2%
160,000	2.48%

160,000 PDE points eventually rescues the run. But notice that it is non-monotonic again, with 40,000 worse than 20,000. And the cost is high. 160,000 PDE points is three times our default, while 500 supervised points is less than half of the default supervised count.

5.3 The minimum-intervention experiment

The previous two sweeps tell us that data helps and collocation helps, but they sweep one axis at a time. The cleaner question is what combination of the two is actually enough. We wanted to isolate which axis is doing the real work. So we ran a grid, varying both the PDE point count and the data point count together, and asked which combinations rescued the run.

This is the experiment that pulled everything together. The two previous sweeps each suggested that more is better, but only this grid tells us which kind of more matters.

Table 7: Rescue map. Which combinations of PDE and data points actually work?

PDE points	Data points	Final error	Rescued?
56,666 (default)	1,133 (default)	42.15%	no
160,000	400	53.82%	no
80,000	500	60.00%	no
20,000	2,000	2.38%	yes
40,000	2,000	3.93%	yes

The pattern is clean. Every rescued run has at least 500 supervised data points. No rescued run has fewer. You can have 160,000 PDE points and still fail if you only have 400 data points. But you can rescue with just 20,000 PDE points if you have 2,000 data points.

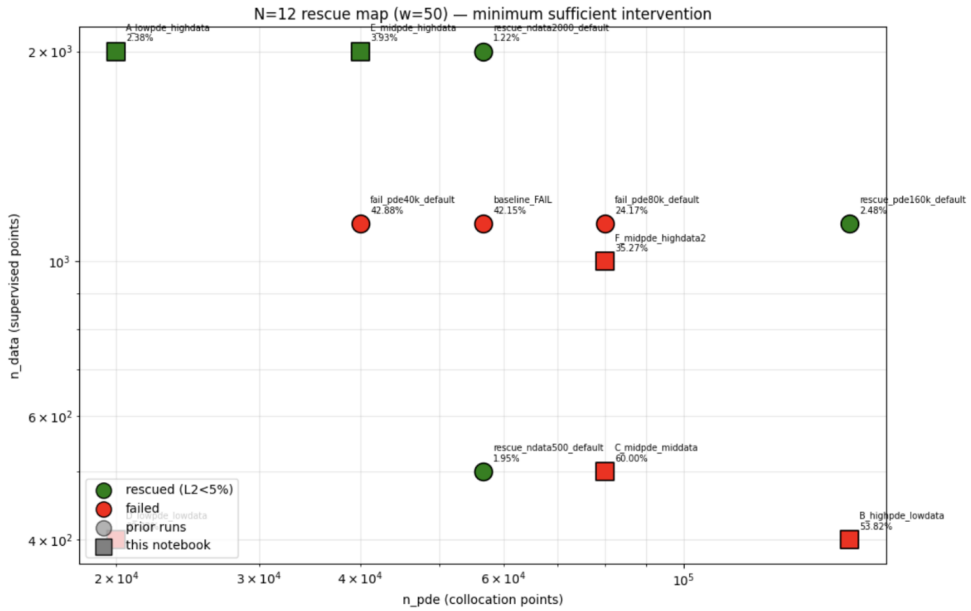

Figure 6: Two-dimensional rescue scatter. Log PDE points on the x-axis, log data points on the y-axis, points coloured by whether the run succeeded. The rescue boundary is nearly horizontal. It is about supervised data, not collocation density.

5.4 Why does data help more than collocation?

Here is the intuition. PDE residuals tell the network that the solution has to satisfy the KdV equation everywhere. But KdV has many solutions. All N-soliton configurations are valid solutions for any N. Enforcing the equation does not tell the network which solution we want.

Supervised data tells the network that at these specific points the solution has these specific values. With twelve localized solitons competing for representational capacity, a handful of pinning constraints at the right places directly tells the network which peak belongs where. It is the difference between saying the equation must hold and saying the answer is this.

6 Three more angles on the same wall

6.1 Does the initial condition matter?

We tried removing the initial condition loss term and replacing it with more supervised data points. The hypothesis was that the initial condition might be redundant if we have enough data anyway.

Table 8: Removing the initial condition versus keeping it.

N	Setup	Final error
3	With IC	0.16%
3	No IC, 400 data points	0.063% (better)
6	With IC	0.49%
6	No IC, 400 data points	0.209% (better)
9	With IC	0.98%
9	No IC, 400 data points	46.0% (catastrophic)

At low N, removing the initial condition and replacing with data actually improves things. The IC term was adding a slightly redundant constraint and creating loss-balance issues. But at N=9, removing the IC is catastrophic. The initial condition at N=9 is the only thing that encodes which of the nine solitons starts where. Data scattered through the domain cannot recover that discrete information.

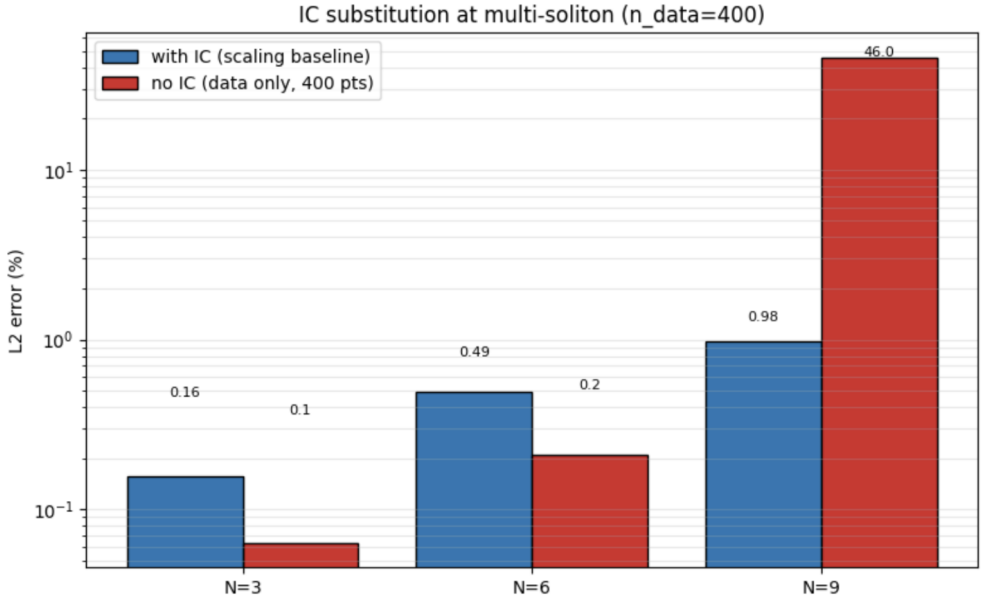

Figure 7: Final L2 error with and without the IC loss term, for N=3, N=6, and N=9. The flip between N=6 and N=9 is the point.

This confirms something we suspected. The IC is not just one more loss term. At high N it is carrying irreducible discrete information about the configuration that nothing else in the training can replace.

6.2 Does noise hurt?

We added Gaussian noise to the supervised data, at 1%, 5%, and 10% of the signal amplitude, and measured how much it degraded N=9.

Table 9: Noise robustness at N=9.

Noise level	Final error	L-BFGS helped?
0%	1.16%	yes
1%	1.29%	yes
5%	5.07%	yes
10%	10.24%	yes

L-BFGS helps at every noise level. This is different from what we saw in our earlier single-soliton work, where L-BFGS sometimes hurt under high noise. The reason is this. In the single-soliton case, Adam was already close to the right basin, so L-BFGS's precision became a liability when the loss surface was noisy. In the multi-soliton case, Adam rarely delivers a clean basin on its own, so L-BFGS has real work to do regardless of noise level.

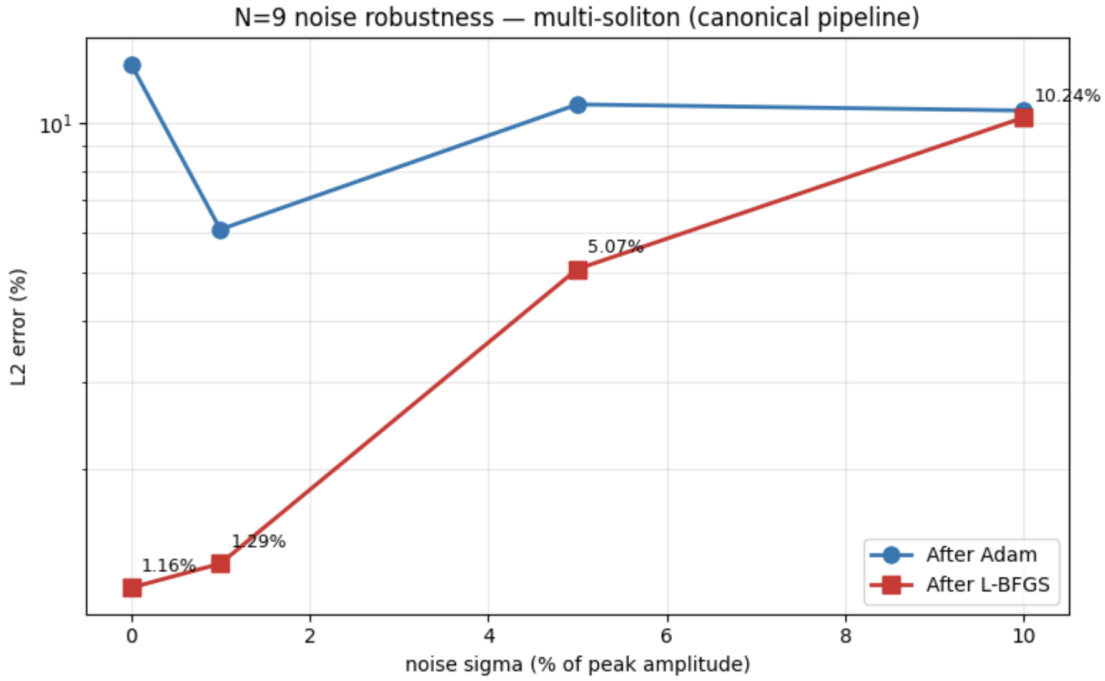

Figure 8: Noise robustness for single-soliton (left) and multi-soliton at N=9 (right). The role of L-BFGS reverses between the two regimes.

6.3 Does context help sharp solitons?

We knew the fastest soliton at N=12, the one at $c = 7$, fails badly. Is that because it is fundamentally hard, or because the configuration around it is hard? We ran a clean test. Take a single soliton at

$c = 4$ in isolation. It fails at 29.5%. Then embed the same soliton in a three-soliton train alongside solitons at $c = 2.0$ and $c = 1.5$. The same soliton, in the same domain, now converges to 0.17%.

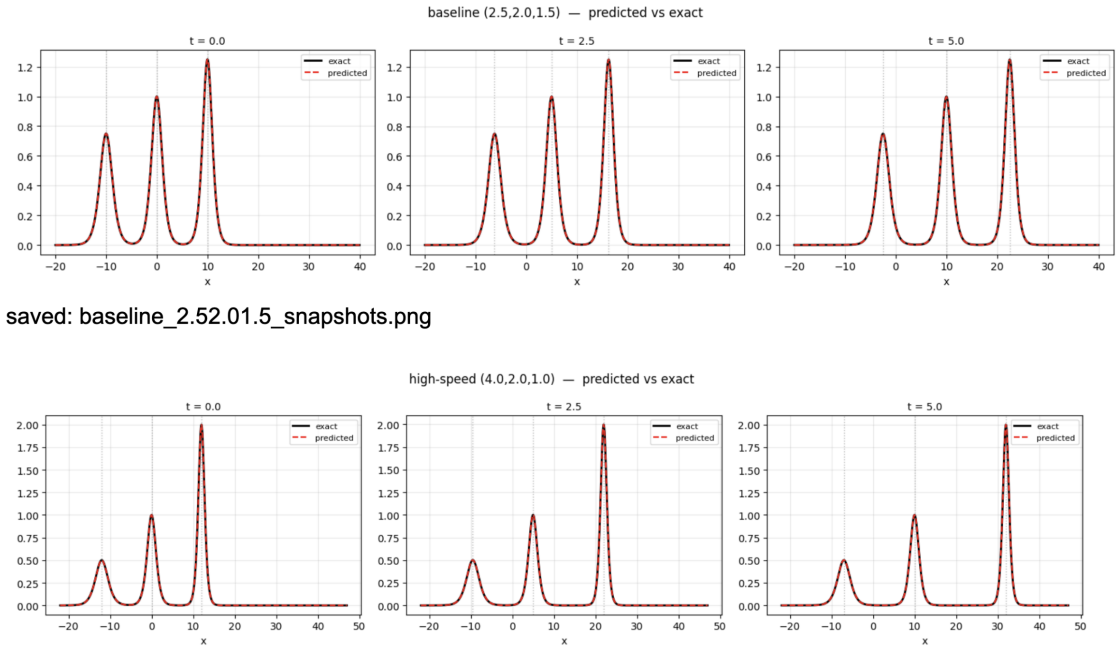

Figure 9: The c=4 soliton in three contexts. Isolated (fails), embedded in a three-soliton train (rescued), and embedded in the twelve-soliton train (fails again).

Context helps, up to a point. When the surrounding configuration gets crowded enough, as it does at N=12, the rescue disappears. The embedding benefit has a ceiling and we have not fully characterized where it is.

7 The reveal: it is not a wall, it is a lottery

After all of those experiments, one question remained. Is the N=12 failure the same every time, or does it vary? In other words, is this a structural problem where every run hits the same wrong basin, or a probabilistic one where different runs land in different bad basins?

We ran the same N=12 configuration eight times, changing only the random seed used to initialize the network weights.

Table 10: Eight seeds, same configuration, six different failure patterns.

Seed	Final error	Worst soliton	Its local error
1	3.11%	10 (c=2.5)	6.95%
7	8.75%	12 (c=1.5)	29.93%
23	29.87%	1 (c=7.0)	60.64%
42	40.59%	2 (c=6.5)	79.36%
99	38.70%	2 (c=6.5)	82.87%
137	19.16%	4 (c=5.5)	48.85%
256	30.61%	3 (c=6.0)	79.88%
999	47.74%	1 (c=7.0)	81.04%

Six different worst solitons across eight seeds. Not the same failure, not the same soliton, not the same pattern. Each seed landed in a different local minimum, a different partial solution that was right for most solitons but wrong for some specific one.

Seed 1 landed at 3.11%. Near-success, without any intervention. Seed 7 landed at 8.75%. Two of eight seeds essentially rescued themselves by chance.

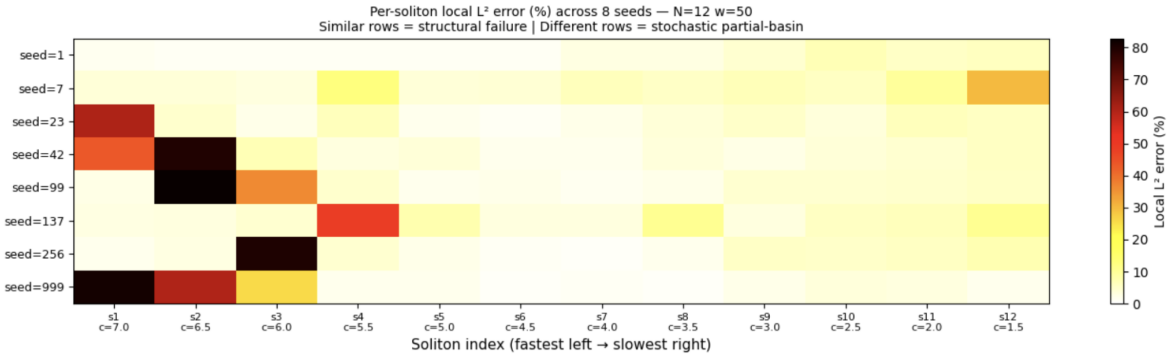

Figure 10: Multi-seed heatmap. Rows are seeds, columns are solitons, colour is local L2 error. Dark patches (high error) scatter across the fast-soliton columns and disappear in the slow-soliton columns. This is the headline figure of the paper.

Looking across seeds, a pattern emerges. The slow solitons are learned correctly regardless of seed. The fast solitons are the lottery. Their errors vary wildly from seed to seed.

Table 11: Slow solitons are deterministic, fast solitons are not.

Soliton	Mean error across seeds	Std devn across seeds
1 (c=7.0)	24.7%	30.2%
2 (c=6.5)	29.7%	35.0%
6 (c=4.5)	1.9%	1.0%
10 (c=2.5)	4.7%	1.2%
12 (c=1.5)	9.0%	8.2%

The picture that comes together is this. The N=12 wall is not a wall at all. It is a probabilistic barrier. The optimizer is navigating a loss landscape that has many near-equivalent local minima, each one a partial solution that gets most solitons right while getting one or two badly wrong. Which local minimum it lands in depends on how it was initialized. And all four rescue mechanisms from the previous two sections have the same underlying interpretation. They reshape the landscape to make the correct full solution more attractive, so the optimizer is less likely to get trapped.

8 Putting it all together

8.1 What the experiments say, in plain terms

The N=12 failure is an optimization landscape problem, not a capacity problem. The network is big enough. The training recipe is fine. The problem is that with twelve fast-moving solitons, the loss function has many near-equivalent partial solutions, and Adam tends to land in one of them rather than finding the true global minimum.

Table 12: What works to rescue N=12, and what it is actually doing.

Intervention	Threshold to rescue	What it is actually doing
More supervised data	500 points	Pins correct values directly, suppresses partial solutions
More collocation	160,000 points	Enforces KdV more strictly, shrinks valid partial solutions
Wider network	w=100 (unreliable)	Smooths the loss surface, but stochastically
Slower speeds	$c_{\max} \leq 4$	Reduces sharpness and landscape complexity
Lucky seed	Occasionally	Landed in the global basin by chance

The cheapest and most reliable fix is supervised data. 500 labelled points, about 0.8% of the PDE collocation point count, gives us a 35-fold error reduction. Every other rescue mechanism costs significantly more.

8.2 The story in one paragraph

We started with a recipe that gave us 0.16% error on three solitons and scaled it as far as it would go. For most of the journey, from N=3 to N=9, it worked better than we expected. At N=12, it failed harder than we expected. The failure turned out not to be what it looked like. Not a capacity ceiling. Not uniform degradation. Not a training bug. It was a probabilistic landscape problem, multiple wrong-but-plausible partial solutions competing with the correct one, and the optimizer not reliably finding the right one. Data anchoring tips the balance.

8.3 What transfers, and what does not

Two kinds of lessons come out of this. Things specific to KdV, like the exact threshold at N=12, the exact rescue values, which solitons happened to fail. And things that are not specific to KdV at all.

The general lessons. Smooth scaling can hide a coming phase transition. N=3 to N=9 looked so clean that N=12 was a genuine surprise. Per-feature error breakdowns are more informative than global error. They reveal whether failure is uniform or partial. Multi-seed runs should be included in any PINN scaling study. Single-seed reports of failed at N=K are reporting one sample from what can be a wide distribution. Data anchoring beats collocation anchoring for landscape problems. And L-BFGS is only as good as the basin Adam delivers to it.

8.4 What we are not claiming

We use linear superposition as our ground truth, not the exact inverse-scattering solution. This is a clean approximation for well-separated solitons but it is not perfect during close approaches. We ran eight seeds for the multi-seed experiment, which is enough to see the qualitative picture but not enough to put precise probabilities on basin sizes. We held the architecture fixed at six layers of 50 neurons for almost everything. The qualitative picture should be robust to these choices, but we would want more seeds and more architectural variation before making sharper quantitative claims.

8.5 Practical takeaway

If you are running PINNs on a multi-feature problem and things look fine up to some N and then break sharply, do not immediately add more layers. First check whether different random seeds give different failure patterns. If they do, you are in an optimization landscape problem, and the cheapest fix is more supervised data, not a bigger network. Also be careful with L-BFGS. It polishes whatever basin Adam delivered. If Adam delivered a wrong basin, L-BFGS will confidently polish that wrong answer. The fix is not to skip L-BFGS. The fix is to fix Adam first.

The diagnostic tools we used, the scaling sweep, the per-feature breakdown, the minimum-intervention map, the multi-seed test, are not KdV-specific. They are a portable methodology for any PINN scaling study. And that portability matters, because the KdV study was never meant to be the final destination.

9 IMP: Why we built all of this, and where it goes next

We wrote this section mainly to keep reminding ourselves what the end goal is and get a very good picture of the road behind and ahead.

9.1 The bigger picture

The KdV equation is the simplest member of a family of shallow-water wave equations. This family is called the b-family. Two other members of the family are particularly important. The Camassa-Holm equation, often written CH, and the Degasperis-Procesi equation, written DP. Both describe shallow water waves at roughly the same level of accuracy as KdV, but with a different balance between dispersion and nonlinearity. The b-family parameter b distinguishes them. CH has $b = 2$, DP has $b = 3$, and KdV is the limit where dispersion dominates.

Why do CH and DP matter? Because their wave solutions look completely different from KdV solitons. And that difference is what makes them hard.

9.2 What is a peakon?

In KdV, the dispersion term u_{xxx} balances wave steepening perfectly, producing smooth rounded soliton bumps. In CH and DP, this balance works differently, and the result is a wave with a sharp peak,

$$u(x,t) = c\, e^{-|x-ct|}. \tag{3}$$

This is called a peakon. It looks like a tent. Continuous, but with a kink at the top. The function itself is fine, its derivative has a jump at the peak. These are not smooth functions. They are the weakest kind of solution that the CH and DP equations admit.

DP goes one step further. When a peakon and an antipeakon collide, they do not pass through each other like KdV solitons do. They annihilate and produce something called a shockpeakon, a solution with a jump discontinuity in u itself, not just in the derivative,

$$u(t,x) = -\frac{1}{t+k}\,\mathrm{sgn}(x)\, e^{-|x|}. \tag{4}$$

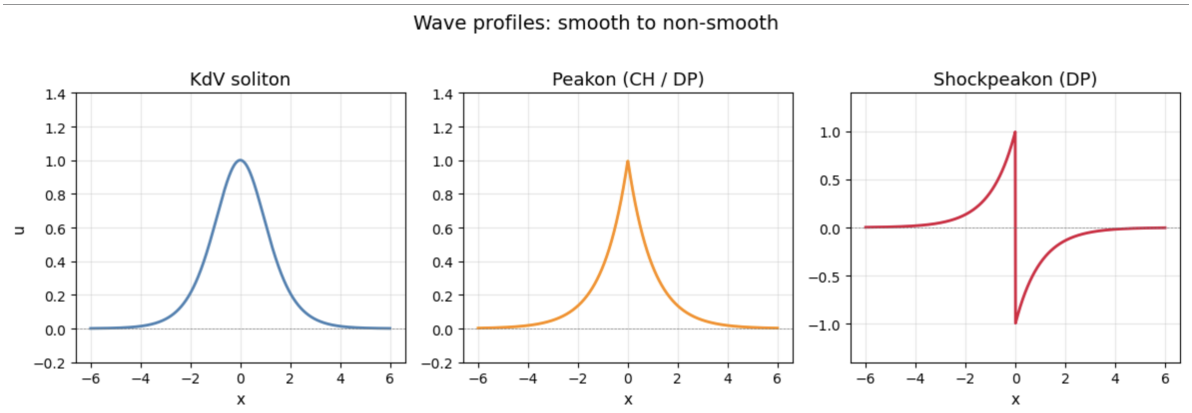

Figure 11: Three wave profiles side by side. A smooth KdV soliton (left), a CH or DP peakon (middle), and a DP shockpeakon (right). The features get progressively less smooth from left to right.

Classical numerical methods struggle badly with peakons. They were designed for smooth functions, and a kink in the derivative breaks their assumptions. Shockpeakons are even harder. They combine a shock, a jump in u, with a peak, a jump in u_x, in a solution that is also evolving over time. Prof. Saha's thesis identifies the shockpeakon problem as genuinely open. No numerical method has successfully simulated it. That is the target our research program is aimed at.

9.3 Is our KdV work actually useful for this?

This is the honest version of the question. PINNs have natural appeal for CH and DP. They are mesh-free, they handle irregular domains, and in principle they can be extended to handle weak non-smooth solutions. But before we can use PINNs for peakons, we need to know how well they actually scale as we add more features to a problem.

That is exactly what we spent this paper answering.

The KdV study gave us four specific tools that we will carry forward.

First, the scaling sweep. We now know what smooth scaling looks like for PINNs on a multi-feature wave problem. We know what a phase transition looks like. When we run the same sweep on CH or DP, we will know whether we are seeing the same kind of failure or something genuinely new.

Second, the per-feature error breakdown. For peakons, the failure mode we expect is different. Errors might concentrate specifically at the peak of each peakon, where the function is non-smooth, rather than spreading across the whole soliton window. The per-feature diagnostic from section 4 adapts directly to this. We will measure error at the peak and in the tail separately rather than just globally.

Third, the minimum-intervention map. Our KdV finding that supervised data is far cheaper than collocation for landscape problems gives us a specific hypothesis for peakons. Data anchored at the peak locations should be disproportionately valuable. We can test this with two data sweeps at the same budget, one with uniform scatter and one with peak-concentrated sampling.

Fourth, the multi-seed diagnostic. For KdV the stochastic versus structural distinction took eight seeds to see clearly. We would expect the same diagnostic to work for CH and DP. Given that peakons are sharper than KdV solitons, we would guess the stochastic regime starts at much lower N. Possibly N=3 or N=4 rather than N=12.

9.4 What we think could be harder in CH and DP

Three things are likely to be different from the KdV setting. We will walk through them in order, starting with the most basic.

The first is a representational challenge that we did not face in KdV. Our tanh network can approximate any smooth function. A peakon is not smooth. It has a kink at its peak. The network will approximate this kink with a sharp-but-smooth fillet. Whether that approximation is good enough to satisfy the CH or DP residual at the peak is an open empirical question. This is the first thing we will test in the b-family work, and it is a question our KdV study could not answer, because KdV solitons are smooth.

That is the first new challenge. The second one is more familiar. Our KdV study found that the optimization landscape became problematic at N=12. Peakons are sharper features than KdV solitons. Closer in character to the $c = 4$ isolated soliton that failed at 29.5% in isolation. Given what we found about sharpness driving landscape difficulty, we predict that the phase transition will appear much earlier for peakons. Perhaps at N=4 to 6 rather than N=12. This is a direct, falsifiable prediction from our KdV findings.

The third challenge is genuinely new territory. At a peakon's peak, the CH and DP equations are satisfied only in a distributional sense. The standard pointwise residual is not well-defined there because the second derivative of u has a delta-function contribution. A naive PINN will compute arbitrary garbage at that point. The right response is to either evaluate the residual only on the smooth regions and impose jump conditions separately, or to reformulate the loss in integrated or weak form. This is methodologically new. Our KdV work did not need it. It will be one of the main architectural challenges of the next phase of the project.

9.5 Why this all fits together

The KdV study is not baseline work we did before getting to the interesting part. It is the diagnostic foundation. We now have a methodology for characterizing how and why PINN scaling breaks. We have a baseline for what smooth scaling looks like, from N=3 to N=9. We have a concrete picture of what landscape-driven failure looks like, from the N=12 multi-seed sweep. We have a tested intervention, data anchoring, that we know works on smooth problems. And we have a set of specific predictions for what will be different in the peakon setting, each prediction traceable to a specific KdV finding.

That is what makes the b-family work possible to do rigorously rather than exploratively. Instead of running CH experiments and hoping something works, we will go in with precise hypotheses and a diagnostic ladder to test them with. The shockpeakon is still a long way off. But the path from here to there is clearer than it was before this paper.

Appendices

A. Full hyperparameter table

Architecture: six hidden layers, 50 neurons per layer, tanh activation, Xavier initialization. Optimizer: 15,000 Adam steps at learning rate 10^{-3}, followed by 2,000 L-BFGS steps with strong Wolfe line search. Loss weights: all four terms equally weighted at 1.0. Canonical seed: 99. Domain rule: $x_{\text{hi}} = x_0^{\max} + 5c_{\max} + 15$. Point scaling: roughly 333 PDE points, 8 IC points, and 7 data points per unit domain width, with 200 boundary points. Time range: $t \in [0, 5]$.

B. Per-soliton diagnostic, formal definition

For a soliton centred at position $x_i(t) = x_{0,i} + c_i t$ at time t, the per-soliton local L2 error is computed over a window of ± 3 units around $x_i(t)$, integrated over all time steps. The window width was chosen so that for our slowest soliton ($c = 1.5$) the window comfortably contains the bulk of the sech^2 profile, and for our fastest soliton ($c = 7.0$) it still contains the bulk of the sharper profile without overlapping its neighbours.

C. A note on stochasticity

The width=100 result reported in section 4 was 0.86% in one run and 38.5% in another, with identical configurations. We have kept the 0.86% number in the main table because it is genuine, but it should be read in light of the multi-seed picture from section 7. Single runs of stochastic experiments are single samples. The fact that one run got lucky does not contradict our conclusion. It illustrates it.

D. b-family reference equations

For convenient cross-reference when reading section 9.

$$\text{KdV:} \quad u_t + 6uu_x + u_{xxx} = 0 \tag{5}$$

$$\text{CH (b=2):} \quad u_t - u_{xxt} + 3uu_x - 2u_x u_{xx} - uu_{xxx} = 0 \tag{6}$$

$$\text{DP (b=3):} \quad u_t - u_{xxt} + 4uu_x - 3u_x u_{xx} - uu_{xxx} = 0 \tag{7}$$

Peakon solution (CH and DP both): $u(t,x) = c\, e^{-|x-ct|}$.

DP shockpeakon: $u(t,x) = -\frac{1}{t+k}\,\mathrm{sgn}(x)\, e^{-|x|}$.